SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4304

Name: Y Jagan
Reg.no: 192373020

Marks: 85/100

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: **Total Marks: 30**

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly. **Questions:**

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
```

```
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I Y Jagan(192373020)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Y JAGAN

1. Online Symposium Portal Given HTML

```
<form action="/register" method="POST">
  <input type="text" name="name">
```

```
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

1. Identify appropriate HTML5 semantic elements missing in the structure.

The form should use semantic HTML5 elements to give the webpage a meaningful structure.

Important missing elements are:

- <header> – contains the title or heading of the symposium.
- <main> – contains the main content of the page.
- <section> – groups related content such as registration.
- <footer> – contains footer information.
- <label> – provides names for form fields and improves accessibility.
- <fieldset> and <legend> – can be used to group related form controls.

The main missing semantic elements are <header>, <main>, <section>, <footer>, <label>, <fieldset>, and <legend>.

2. Modify the structure to make it standards-compliant.

A better and standards-compliant version is:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Online Symposium Registration</title>

</head>

<body>

<header>

  <h1>Online Symposium</h1>

</header>

<main>

  <section>
    <h2>Student Registration</h2>

    <form action="/register" method="POST">
```

```
        <label for="name">Name:</label>
        <input type="text" id="name" name="name" required>
        <br><br>
        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required>
        <br><br>
        <button type="submit">Submit</button>
      </form>
    </section>
</main>
<footer>
    <p>&copy; 2026 Online Symposium</p>
</footer>
</body>
</html>
```

Improvements made:

1. Added <!DOCTYPE html> to declare HTML5.
2. Added lang="en" to specify the language.
3. Added <meta charset="UTF-8">.
4. Added viewport settings for responsive design.
5. Added semantic elements.
6. Added <label> elements for accessibility.
7. Added id attributes to connect labels with inputs.
8. Added required for form validation.
9. Specified type="submit" for the button.

3. Interpret the HTTP request generated during form submission.

When the student clicks **Submit**, the browser sends an HTTP POST request to the server.

For example:

POST /register HTTP/1.1

Host: example.com

Content-Type: application/x-www-form-urlencoded

name=Jyoshna&email=jyoshna@example.com **Explanation:**

- POST – HTTP method used to send data.
- /register – server URL where the form data is sent.
- Host – identifies the server.
- Content-Type – specifies the format of the submitted data.
- The request body contains the form values.

The server receives this information and processes the student's registration.

4. Analyze the HTTP response cycle from server to browser.

The HTTP response cycle works as follows:

Step 1: Student fills in the form.

Step 2: Student clicks the Submit button.

Step 3: Browser creates an HTTP POST request.

Step 4: The request is sent to the /register server.

Step 5: The server receives and processes the registration data.

Step 6: The server sends an HTTP response.

Step 7: The browser receives the response and displays the result to the student

Simple flow:

User

↓

HTML Form

↓

HTTP POST Request

↓

Web Server

↓

Process Registration

↓

HTTP Response

↓

Browser

↓

Display Result **5.**

Suggest improvements for usability and accessibility.

The following improvements can be made:

1. Use <label> for every input field.
2. Use required for mandatory fields.
3. Use meaningful placeholder text where appropriate.
4. Provide clear error messages for invalid input.
5. Use proper heading levels such as <h1> and <h2>.
6. Make the form keyboard accessible.
7. Use sufficient color contrast.
8. Use responsive design so the form works on mobile devices.
9. Use appropriate input types such as email.
10. Provide clear instructions to users.

Conclusion: These improvements make the portal easier to use and accessible to users with different abilities.

2. Cross-Browser Compatibility Issue Given

HTML

```
<html>
  <head>
    <title>Company</title>
  </head>
```

```
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

1. Identify improper nesting and missing closing tags.

There are some HTML structure problems.

Problems:

1. <h1> is not properly closed.
2. <p> is not properly closed.
3. The <p> element is inside the <h1> element because the <h1> closing tag is missing.
4. Semantic elements such as <header> and <section> are missing.
5. <!DOCTYPE html> is missing.

Correct closing tags:

```
<h1>Welcome</h1>
<p>About us</p>
```

2. Analyze how different browsers may interpret this structure.

Different browsers use HTML parsing rules to correct invalid HTML automatically.

For example, if closing tags are missing, a browser may automatically insert them or change the document structure internally.

Because browsers may handle malformed HTML differently:

- The page layout may differ.
- Text may appear in unexpected locations.
- CSS selectors may behave differently.
- Accessibility tools may interpret the page incorrectly.

Therefore, properly written HTML should always be used for consistent browser behavior.

3. Identify missing semantic elements.

The following semantic elements can be added:

- <header> – for the company heading.
- <main> – for the main page content.
- <section> – for the About Us content.
- <footer> – for footer information.

Instead of using only , semantic HTML provides meaning to the content.

4. Provide a corrected W3C-compliant HTML structure.

```
<!DOCTYPE html>
<html lang="en">

<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Company</title>
</head>

<body>

<header>
   <h1>Welcome</h1>
</header>

<main>
   <section>
      <h2>About Us</h2>
      <p>We are a professional company.</p>
   </section>
</main>

<footer>
   <p>&copy; 2026 Company</p>
</footer>

</body> </html>
```

This structure follows proper HTML5 syntax and uses semantic elements.

5. Suggest techniques to ensure cross-browser compatibility.

The following techniques can be used:

1. Use valid HTML5 syntax.
2. Always include <!DOCTYPE html>.
3. Properly close all HTML tags.
4. Use semantic HTML elements.
5. Use CSS resets or normalization when required.
6. Avoid browser-specific features unless necessary.
7. Test the website in Chrome, Firefox, Edge, and Safari.
8. Use responsive CSS.
9. Use feature detection instead of assuming browser support.
10. Validate HTML and CSS using standard validation tools.

Conclusion: Following web standards and testing on multiple browsers helps provide a consistent user experience.

3. Form Submission Failure Given

HTML

```
<form action="/submit" method="GET">
   <input type="text" name="username">
   <input type="password" name="password">
   <button type="button">Login</button>
</form>
```

1. Identify issues in the form configuration.

There are two major problems.

Problem 1: Button type

The button is:

<button type="button">Login</button> type="button"

does not submit the form**.**

Therefore, clicking the button does not automatically generate the form submission request.

It should be:

<button type="submit">Login</button> Problem
2: GET method

The form uses: method="GET"

GET places the form data in the URL.

For example:

/submit?username=Jyoshna&password=12345

This is not appropriate for login credentials.

2. Analyze why the HTTP request is not triggered. The

HTTP request is not triggered because the button has:

type="button"

A normal button does not submit the form.

The browser submits a form when:

<button type="submit"> is used or when an equivalent

submit control is activated.

Therefore, clicking the current Login button does not send the form data to /submit.

3. Interpret the use of the GET method in this scenario.

The GET method sends form data as part of the URL.

For example:

GET /submit?username=Jyoshna&password=12345 HTTP/1.1 Problems
with using GET for login:

- Password can appear in the URL.
- URLs may be stored in browser history.
- URLs may appear in logs.
- URLs may be exposed through other mechanisms.

Therefore, GET should not be used for submitting passwords or other sensitive information.

POST is more appropriate for login forms.

4. Propose a corrected implementation.

A corrected login form is:

```
<!DOCTYPE html>
<html lang="en">

<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Login</title>
</head>

<body>

<form action="/submit" method="POST">

   <label for="username">Username:</label>
   <input type="text" id="username" name="username" required>

   <br><br>

   <label for="password">Password:</label>
   <input type="password" id="password" name="password" required>

   <br><br>

   <button type="submit">Login</button>

</form>

</body>
</html>
```

Changes made:

- Changed GET to POST.

- Changed type="button" to type="submit".
- Added labels.
- Added id attributes.
- Added required validation.

5. Suggest improvements for secure data transmission.

For secure login transmission:

1. Use POST instead of GET for passwords.
2. Use HTTPS to encrypt data between browser and server.
3. Never store passwords as plain text.
4. Hash passwords securely on the server.
5. Validate and sanitize user input.
6. Use secure session management after login.
7. Use appropriate authentication and authorization mechanisms.
8. Do not include passwords in URLs.
9. POST alone does not make a login secure. HTTPS is required to protect the data during transmission.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	7.5	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	7.5	Detailed, wellexplained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	6	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	4.5	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	3	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	1.5	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand
TOTAL	**22/30**				